MNEMORAG: Exploiting Inter-Request Correlations for High-Performance RAG Serving

Xin Yan, Hang Huang, Zhuo Huang, Song Wu, Hai Jin
National Engineering Research Center for Big Data Technology and System,
Services Computing Technology and System Lab, Cluster and Grid Computing Lab,
School of Computer Science and Technology, Huazhong University of Science and Technology,
Wuhan, China
{lemonyx, hanghuang, huangzhuo, wusong, hjin}@hust.edu.cn

***Abstract*—Retrieval-Augmented Generation (RAG) grounds large language models (LLMs) in external knowledge, but its practical deployment is hindered by substantial serving overheads from expensive vector retrieval, long-context prefill computation, and the strict serialization of retrieval and generation. Existing approaches primarily optimize the serving pipeline on a per-request basis, leaving inter-request correlations underexploited. In this paper, we present MNEMORAG, a correlation-aware RAG serving system that systematically exploits inter-request correlations to jointly optimize retrieval and generation. MNEMORAG introduces *similarity-aware retrieval*, which leverages historical query–result associations to dynamically narrow the search space or directly reuse retrieval results, thereby eliminating redundant vector searches. It further employs *hotness-driven prefix caching*, which exploits skewed document popularity and prompt-order flexibility to maximize KV-cache reuse and reduce prefill computation. Finally, MNEMORAG adopts *asynchronous RAG scheduling* to decouple retrieval and generation across requests, enabling cross-stage overlap and improving resource utilization. Extensive evaluations across diverse RAG workflows and widely used datasets show that MNEMORAG consistently outperforms state-of-the-art systems, delivering $1.26\times$–$2.46\times$ end-to-end speedups with only a 1.1–2.5-point recall drop and comparable answer quality. Source code is available at: https://github.com/Lemonandyx/MnemoRAG**

***Index Terms*—RAG, Correlation, KV Cache, Scheduling**

I. INTRODUCTION

Large Language Models (LLMs) have become a foundational technology in modern natural language processing, demonstrating strong capabilities in reasoning, tool use, natural language understanding, and generation [1]. Built on transformer architectures and trained on large-scale datasets, LLMs encode rich linguistic and factual knowledge within their parameters [2], enabling accurate and context-aware inference for a wide range of tasks [3], [4]. However, reliance solely on *parametric knowledge* can become a limitation in knowledge-intensive applications (e.g., medical diagnostics, financial fraud detection, and legal research [5]–[7]) that demand precise factual grounding. In such scenarios, LLMs may produce hallucinations [8], where outputs are factually incorrect or fabricated, thereby compromising reliability.

To address this challenge, *Retrieval-Augmented Generation* (RAG) [9] has emerged as a practical approach that augments LLMs with external, domain-specific knowledge to improve factual accuracy [10], [11]. As shown in Figure 1,

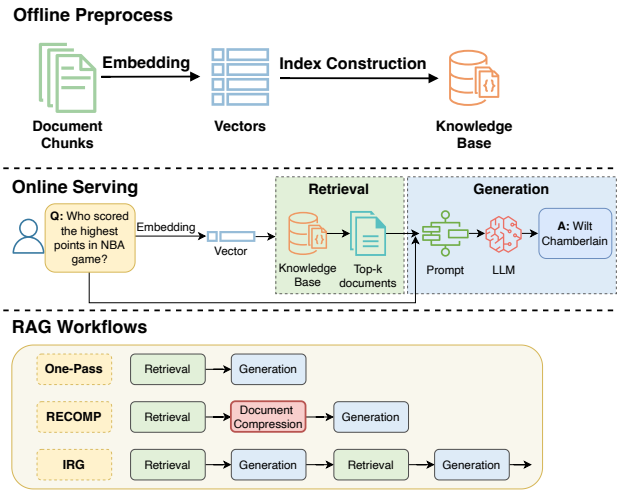


Fig. 1. RAG system overview.

RAG typically follows a two-stage serving pipeline in which **retrieval precedes generation**. During retrieval, similarity search is performed over a preprocessed knowledge base, such as a vector database, to identify the top-k documents most relevant to the user query. These documents are then combined with the query to form an *augmented prompt* for LLM inference [12]. By grounding generation in retrieved evidence, RAG mitigates hallucinations without costly task-specific fine-tuning, making it a broadly applicable paradigm for knowledge-intensive applications [13]–[15].

However, despite its effectiveness, RAG reshapes the serving pipeline by introducing a latency-critical retrieval path and expanding the computational footprint of generation. Consequently, RAG serving often incurs markedly higher end-to-end latency and lower throughput than retrieval-free LLM serving, posing a major barrier to deployment at scale [16], [17]. During retrieval, similarity search algorithms such as *Approximate Nearest Neighbor Search* (ANNS) are executed over large-scale vector indices (e.g., IVF [18]); such operations are often CPU-intensive and can contribute considerable latency [19]–[21]. The retrieved documents further expand the input context, increasing the computation and memory-access

overhead of the LLM prefill stage[1] [22], [23]. More importantly, the strict inter-stage dependency places retrieval and generation on a sequential critical path, limiting opportunities for execution overlap and amplifying latency [15], [16], [24], [25]. Collectively, these overheads create a fundamental trade-off between knowledge augmentation and serving efficiency, necessitating cross-stage optimization for RAG serving.

Recent studies have explored stage-wise and system-level optimizations across the retrieval–generation pipeline. In the retrieval stage, HaS [26] validates speculative retrieval from restricted scopes through query homology, while Proximity [27] reuses cached document results for similar queries. GPTCache [28] caches query-response pairs and reuses stored responses for semantically similar requests based on a configurable global similarity threshold, while QVCache [29] caches retrieved vectors and validates ANN cache hits using adaptive regional thresholds. In the generation stage, CacheBlend [30] and Cache-Craft [31] reuse KV caches with selective recomputation to reduce redundant prefill computation. RAGCache [22] caches the KV states of retrieved documents, while ContextPilot [32] improves prefix-cache reuse by reorganizing overlapping contexts. Beyond stage-specific optimizations, RAGO [33] optimizes task placement, batching, and resource allocation; HedraRAG [34] coordinates retrieval and generation across heterogeneous resources; and FlashRAG [17] provides a modular framework for constructing and evaluating complete RAG pipelines. However, these approaches exploit cross-request correlations only partially, including shared retrieval results, reusable generation states, and overlapping contexts. This lack of holistic coordination limits resource efficiency and leaves substantial performance gains unexplored in multi-request serving.

By empirically analyzing inter-request correlations across widely used open-domain QA benchmarks, including NQ [35], HotpotQA [36], SQuAD [37], and 2WikiMultiHopQA [38], we identify three key opportunities for improving RAG serving. First, semantically similar requests often retrieve highly overlapping document sets and even identical top-k results, enabling retrieval result reuse to eliminate redundant similarity searches. Second, document accesses exhibit strong skewness, where a small subset of documents dominates retrieval frequency, while generation quality remains largely insensitive to document ordering. This property enables hotness-aware KV cache management to improve prefix cache hit rates and reduce redundant prefill computation. Third, although retrieval and generation are sequentially dependent within individual requests, such dependency does not exist across requests, allowing decoupled scheduling to reduce pipeline serialization and improve overall serving efficiency.

Based on these observations, we propose MNEMORAG, a high-performance RAG system that exploits inter-request correlations to jointly optimize retrieval and generation. (1) To eliminate redundant retrieval computation, MNEMORAG introduces *similarity-aware retrieval* (§IV-B), which leverages historical query–retrieval correlations to dynamically adjust the search strategy. By maintaining a lightweight *Query Base* of previous queries and retrieval results, MNEMORAG either reuses existing results for highly similar queries or narrows the search space for related ones. (2) To accelerate generation, MNEMORAG presents *hotness-driven prefix caching* (§IV-C), which exploits document popularity skewness and prompt-order flexibility to maximize KV cache reuse. It prioritizes frequently accessed documents to improve prefix sharing and employs hotness-aware cache management to retain valuable KV states. (3) To overcome pipeline serialization, MNEMORAG introduces *asynchronous RAG scheduling* (§IV-D), which decouples retrieval and generation across requests, enabling their overlap and improving resource utilization.

We implement MNEMORAG with approximately 3K lines of code atop Faiss [18] and vLLM [39]. Extensive evaluations across diverse RAG workflows and widely used datasets show that MNEMORAG outperforms state-of-the-art baselines, achieving $1.26\times$–$2.46\times$ speedups with only a 1.1–2.5-point recall drop and comparable answer quality. These results establish exploiting inter-request correlation as an effective systems-level optimization for scalable RAG serving.

The key contributions of this paper are as follows:

- We characterize the performance of RAG serving under open-domain workloads and uncover three optimization opportunities arising from inter-request correlations: retrieval-result overlap, skewed document popularity, and cross-request pipeline independence.
- We design and implement MNEMORAG, a correlation-aware high-performance RAG serving system that jointly optimizes retrieval and generation through similarity-aware retrieval, hotness-driven prefix caching, and asynchronous RAG scheduling.
- We evaluate MNEMORAG on widely used RAG workloads, delivering substantial end-to-end speedups while largely preserving retrieval recall and generation quality.

II. BACKGROUND

This section characterizes the architecture and performance bottlenecks of RAG serving. Section II-A presents the end-to-end serving workflow, while Sections II-B and II-C identify vector search and prefill computation as the dominant latency sources in retrieval and generation, respectively. These observations motivate exploiting inter-request correlations for end-to-end serving optimization.

A. RAG System and Serving Workflow

A standard *Retrieval-Augmented Generation (RAG)* system consists of tightly integrated retrieval and generation modules that collectively enable knowledge-enhanced generation [9], [40]. The retrieval module maintains a vectorized knowledge base in which documents are encoded into high-dimensional embeddings and partitioned into semantically coherent chunks, and is integrated with high-performance vector search libraries

[1]The prefill stage of LLM inference processes all input tokens to construct the key-value (KV) cache before autoregressive decoding begins.

such as Faiss [18]. It supports efficient indexing, scalable similarity search, and retrieval of semantically relevant documents in response to arbitrary queries [41]–[43]. The generation module uses a large language model (LLM), deployed through serving engines such as vLLM [39] or SGLang [44], to produce responses conditioned on the query and retrieved context [14]. Modern serving engines incorporate advanced optimizations including prefix caching and sequence batching to achieve low-latency, high-throughput inference, enabling seamless coordination between retrieval and generation for interactive, multi-turn applications [23], [45], [46].

Retrieval and generation form the core execution pipeline of RAG serving. As illustrated in Figure 1, conventional RAG systems typically adopt a one-pass retrieve–then–generate workflow, in which each query triggers a single retrieval operation whose results are directly incorporated into the generation prompt. Although simple and efficient, this workflow is highly dependent on retrieval quality and is susceptible to the *lost-in-the-middle* effect when the retrieved context is noisy or excessively long [47]. To reduce long-context processing overhead while preserving salient evidence, retrieve–compress–generate workflows, exemplified by RECOMP-style methods [48]–[51], insert an intermediate compression or refinement stage before generation. In contrast, iterative retrieval–generation (IRG) workflows [36], [52]–[54] repeatedly interleave retrieval and generation, using intermediate outputs, such as refined queries or reasoning traces, to guide subsequent retrieval and improve the supporting context. Collectively, these workflow variants enhance the efficiency and robustness of RAG under long or noisy contexts, while introducing increasingly heterogeneous stage compositions and inter-stage dependencies that demand workflow-aware serving optimization.

B. Retrieval via Vector Search

Retrieval is typically performed via vector search, which identifies query-relevant embeddings from a large-scale vectorized knowledge base. The input query is first encoded into a high-dimensional vector based on the embedding model (e.g., 768 dimensions for BERT [41] and 1024 for E5-Large [42]). The system then applies the nearest neighbor search (NNS) over the embedding space to retrieve the top-k vectors based on similarity metrics such as cosine similarity, inner product, or Euclidean distance [55]. Retrieval quality is measured by Recall@k, defined as the average fraction of the exact top-k nearest neighbors recovered by approximate search. The retrieved vectors are subsequently mapped to their corresponding documents and supplied to the generation stage.

As knowledge bases continue to grow, exact nearest-neighbor search becomes prohibitively expensive because it requires exhaustive distance evaluations over millions or billions of vectors [12]. Practical RAG systems therefore rely on approximate nearest-neighbor search (ANNS) techniques [18], [56]–[58], such as IVF [18] and HNSW [56], which substantially accelerate retrieval at the cost of a small, controllable loss in recall. Among them, IVF is widely adopted because of its scalability and efficiency. IVF partitions the embedding space via coarse quantization, grouping vectors around learned centroids during index construction [59], and structures query processing as a two-stage pipeline. A coarse search first selects the top-$nprobe$ nearest clusters, followed by a fine-grained search over vectors within these clusters. Practical systems further batch multiple queries during retrieval, with the *retrieval batch size* (RBS) controlling the number of queries processed concurrently. Thus, $nprobe$ governs the trade-off between search accuracy and computational cost, whereas RBS determines the degree of parallelism [57].

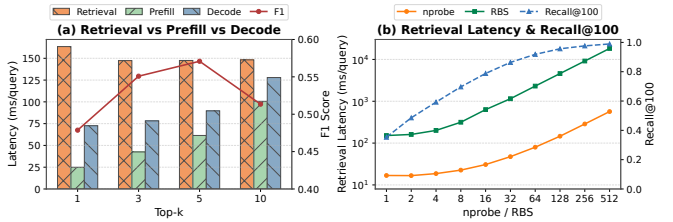


Fig. 2. Retrieval latency characterization. (a) Stage-wise latency across varying top-k. (b) Impact of IVF parameters on retrieval latency and quality.

We conduct an empirical study on the NQ [35] dataset to characterize the latency breakdown of RAG serving. We set the request rate to 1 req/s and vary top-k under a representative IVF configuration ($nprobe = 128$, RBS $= 256$) to characterize the retrieval latency due to vector search. As shown in Figure 2(a), retrieval latency is largely insensitive to top-k, indicating that the dominant cost comes from index traversal rather than the number of retrieved results. Consequently, retrieval contributes substantially to end-to-end latency and *Time to First Token* (TTFT)[2]. Meanwhile, F1 increases from 0.479 at top-$k = 1$ to 0.571 at top-$k = 5$, but drops to 0.513 at top-$k = 10$, indicating that excessive retrieval increases generation cost while exacerbating contextual distraction and the *lost-in-the-middle* effect. We fix top-k to 3 for latency measurements and independently vary $nprobe$ and RBS, while evaluating Recall@100 across different $nprobe$ settings. Figure 2(b) shows that increasing $nprobe$ improves Recall@100 by exploring more clusters but introduces search latency, revealing an inherent quality–efficiency trade-off. In contrast, increasing RBS improves retrieval throughput by amortizing computation and memory access costs while increasing individual query latency due to batching delays. Overall, these results establish retrieval as a major performance bottleneck and highlight the need for coordinated optimization of top-k, $nprobe$ and RBS to balance retrieval quality, latency, and throughput.

C. Generation with Retrieved Documents

A fundamental distinction between RAG serving and conventional LLM serving lies in their prompt construction paradigms. Instead of directly processing user queries, RAG systems construct an *augmented prompt* by integrating retrieved documents with system instructions and the original query. This retrieval-augmented context introduces substantial

[2]TTFT captures the latency from request arrival to the first output token, including retrieval and prefill overheads before decoding.

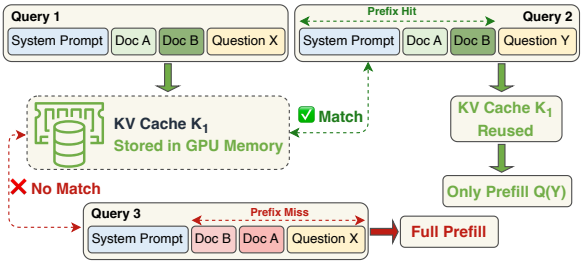


Fig. 3. Prefix KV cache reuse under exact matching. Matching prefixes enable partial prefill via KV cache reuse, while mismatches incur full prefill.

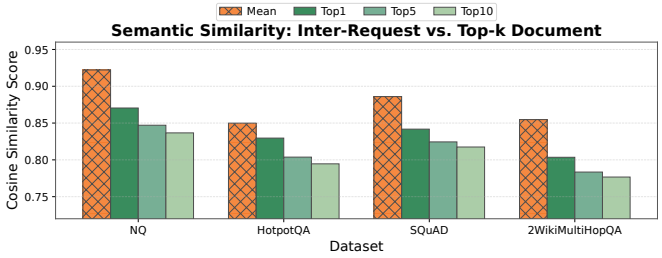


Fig. 4. Mean inter-request semantic similarity versus top-k retrieval similarity.

variability in prompt length and composition, making generation efficiency highly sensitive to retrieval context. Since LLM serving processes the entire input context during prefill before autoregressive decoding, longer RAG prompts can significantly increase prefill computation [60]. To mitigate such overhead, modern LLM serving systems employ KV-cache management and reuse techniques.

Among these techniques, prefix caching extends KV-cache reuse across requests by reusing previously computed KV states when requests share identical prefixes. This approach avoids redundant prefill computation and is particularly promising for RAG workloads, where requests often share system prompts and overlapping retrieved content [22], [44], [61], [62]. However, its effectiveness fundamentally depends on strict and exact token-level prefix matching across requests. As KV states are deterministically derived from the entire prefix sequence, including token identities and positional encodings [45], even minor prompt variations can cause prefix mismatches and limit cache reuse.

This limitation becomes more pronounced in RAG systems, where prompt construction is tightly coupled with retrieval outputs. As illustrated in Figure 3, KV cache reuse is only possible when requests retrieve identical documents in the same order. Any variation in document content or ordering breaks prefix matching and invalidates cached KV states. Existing serving engines [17], [39], [44] provide limited support for such retrieval-induced prompt variability, resulting in low prefix KV cache hit rates for RAG workloads. As shown in Figure 2(a), this limitation directly translates into increased prefill latency. Our measurements (Figure 14) further demonstrate that increasing top-k sharply reduces prefix matching rates (e.g., from 20.35% at top-1 to 2.84% at top-10).

III. Motivation

In this section, we characterize inter-request behaviors in RAG serving and uncover three optimization opportunities: semantic correlation among requests, skewed document popularity, and cross-request execution independence. These observations uncover untapped optimization opportunities and drive the design of MnemoRAG.

A. Inter-request Semantic Similarity

Existing RAG systems perform retrieval independently for each request, treating queries in isolation [16], [17]. The associated similarity computation over large-scale vector spaces remains expensive, despite advances in vector search techniques. When requests exhibit semantic correlations, they often access overlapping regions of the embedding space and retrieve similar document sets. However, existing systems overlook such cross-request redundancy, leading to repeated similarity searches and unnecessary retrieval overhead.

To characterize this opportunity, we analyze several open-domain RAG workloads and uncover substantial inter-request semantic similarity arising from recurring information-seeking patterns. As illustrated in Figure 4, queries exhibit strong semantic alignment with their retrieved top-k documents. More importantly, the mean similarity between each query and its globally nearest semantic counterpart within the workload exceeds 0.85, surpassing the query–document similarity. These results demonstrate that semantic locality is an intrinsic property of RAG workloads rather than an incidental artifact. Such locality implies that queries need not be retrieved in isolation: semantically related historical requests provide useful priors for locating relevant documents, reducing redundant exploration of the vector index. This motivates *similarity-aware retrieval* in MnemoRAG, which translates inter-request semantic locality into lower retrieval cost.

B. Document Access Skewness

The effectiveness of prefix caching is a key factor determining RAG serving efficiency, as it directly reduces redundant prefill computation during generation. However, achieving high cache utilization in RAG is challenging because cached KV states can only be reused when future requests exhibit sufficient prefix overlap with previous prompts. Unlike conventional LLM workloads, RAG prompts are dynamically constructed from retrieved documents, whose access patterns exhibit strong document-level locality. Existing serving engines primarily rely on token-level prefix matching and generic cache management policies, such as least-recently-used (LRU) or first-come-first-served (FCFS) [39], [44], limiting their ability to exploit document-level reuse opportunities in RAG workloads. As a result, frequently accessed documents may fail to contribute to effective prefix reuse, leaving substantial optimization opportunities unexplored.

We analyze several open-domain RAG workloads and observe pronounced skewness in document access patterns. As shown in Figure 5, document accesses are highly concentrated on a small subset of frequently retrieved documents, indicating substantial document-level locality across requests. However,

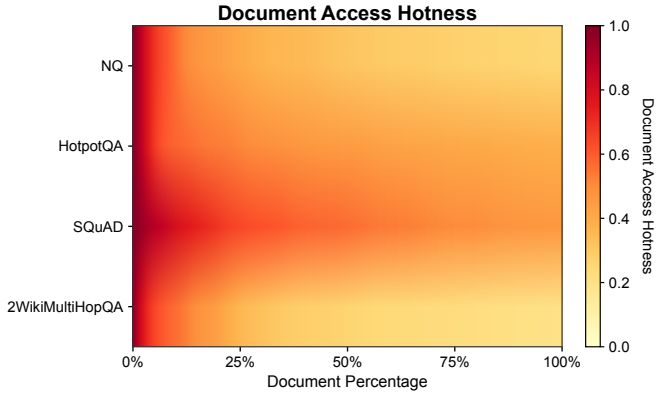


Fig. 5. Document access distribution.

existing prefix caching mechanisms fail to fully exploit this locality due to their strict token-level matching requirement: even minor variations in prompt construction can invalidate cached KV states [63], [64]. In particular, retrieved documents are typically concatenated according to retrieval order, which may vary across requests and cause prompts with highly overlapping content to diverge [9].

This observation reveals a key opportunity: retrieved documents form an unordered semantic context whose effectiveness depends on information coverage rather than positional arrangement. Our empirical analysis further shows that reordering documents within the same retrieved set using a lightweight *hint* mitigates accuracy degradation. This insight motivates *hotness-driven prefix caching* in MNEMORAG, which leverages document access skewness to improve prefix KV cache reuse and reduce redundant prefill computation.

C. Independent Inter-request Execution

Retrieval must precede generation within each request because the generated response depends on the retrieved context. This dependency is request-local: retrieval for a subsequent request is independent of generation for the current one. However, frameworks such as LangChain [16] and FlashRAG [17] often enforce stage-synchronous scheduling across request boundaries, deferring subsequent retrieval until the current generation stage completes. Propagating an intra-request dependency into the inter-request schedule serializes independent stages, creates pipeline bubbles, and places retrieval and generation latencies additively on the end-to-end critical path.

Eliminating this barrier is insufficient to ensure latency-effective pipelining. Preparation latency is fully concealed only when each subsequent request becomes generation-ready within the ongoing generation window; otherwise, the uncovered preparation tail delays the next generation stage. HedraRAG [34] mitigates this effect through fine-grained speculative pipelining. Nevertheless, retrieval extending beyond the speculative window remains on the critical path, while incorrect speculation introduces correction or regeneration overhead. Pipeline effectiveness is therefore determined by whether the complete and valid preparation path can be accommodated within the available generation interval.

Satisfying this is more challenging in MNEMORAG, whose preparation path includes query processing, vector search, retrieval-path selection, and the restoration of reusable prefix KV states from host memory [39]. These heterogeneous stages consume CPU, GPU, and interconnect resources and may contend with active generation for shared hardware capacity. Serial execution leaves their aggregate latency exposed, whereas unconstrained overlap may increase generation latency through resource contention. The scheduler must therefore exploit inter-request independence while preserving intra-request dependencies and controlling interference with ongoing generation. These requirements motivate the resource-aware *asynchronous RAG scheduling* in MNEMORAG.

IV. Design and Implementation

In this section, we present MNEMORAG, a correlation-aware RAG serving system that improves end-to-end efficiency by exploiting inter-request correlations and workload-level locality. Unlike conventional RAG systems that treat requests as independent execution units, MNEMORAG leverages cross-request redundancy to jointly optimize retrieval, prefix caching, and execution scheduling. The retrieval component is implemented with approximately 2.5K lines of code on top of Faiss (v1.8.0) [18] for scalable vector search, while the generation component adds around 0.5K lines of code on top of vLLM (v0.11.2) [39] for efficient LLM serving.

At a high level, MNEMORAG follows three design principles. (1) It exploits semantic correlations across requests to eliminate redundant similarity computation during retrieval. (2) It leverages document access locality to maximize prefix sharing and improve KV cache reuse during generation. (3) It decouples retrieval and generation across requests to enable cross-stage concurrency and improve GPU utilization.

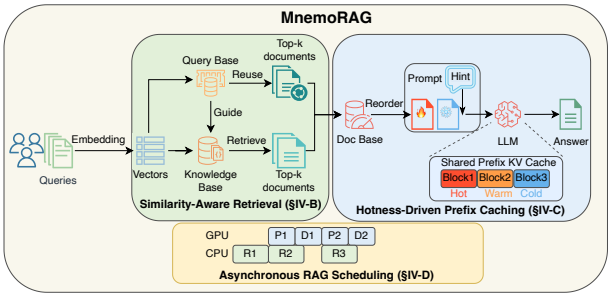


Fig. 6. MNEMORAG Architecture.

A. MNEMORAG Overview

The overall architecture of MNEMORAG is illustrated in Figure 6. Given an input query, MNEMORAG executes a correlation-aware serving workflow that jointly optimizes retrieval, generation, and cross-request scheduling.

First, MNEMORAG performs *similarity-aware retrieval* by leveraging historical query–retrieval correlations maintained in a *Query Base*. Based on the similarity with previous queries, the system either directly reuses existing retrieval results or narrows the vector search space to relevant candidate clusters, thereby reducing redundant similarity computation.

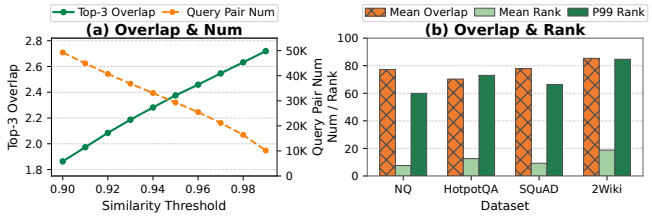


Fig. 7. Semantic similarity–induced overlap in retrieval. (a) Top-k overlap versus similarity. (b) Probe overlap and rank statistics.

Second, before LLM prefill, MNEMORAG applies *hotness-driven prefix caching* by exploiting document access locality maintained in a *Doc Base*. The system reorders retrieved documents according to their popularity to maximize shared prompt prefixes across requests. This reordering improves prefix KV cache reuse, while a lightweight *hint* mitigates the resulting quality degradation.

Finally, MNEMORAG employs *asynchronous RAG scheduling* to overlap subsequent request preparation with generation. Once the current microbatch enters prefill, CPU retrieval proceeds across prefill and decoding, while GPU retrieval is selectively admitted and CPU-resident KV blocks are restored asynchronously. This one-microbatch look-ahead reduces exposed latency while preserving intra-request dependencies.

B. Similarity-Aware Retrieval

Vector search in RAG retrieval typically follows a two-stage ANN pipeline. It first identifies the top-$nprobe$ nearest clusters, forming an ordered probe set $C = \{c_1, c_2, \ldots, c_{nprobe}\}$, and then performs fine-grained search within these clusters to retrieve the top-k documents. Although this hierarchical design avoids exhaustive search over the entire corpus, probing multiple clusters remains a significant retrieval overhead.

Prior similarity-based systems, including GPTCache [28], Proximity [27], and QVCache [29], primarily use query similarity to determine cache reuse, thereby avoiding redundant retrieval or generation on cache hits. However, semantic similarity provides richer optimization opportunities beyond cache hits. MNEMORAG introduces *similarity-aware retrieval*, which leverages inter-request similarity as a continuous control signal to dynamically adjust retrieval computation, including probe budgets and candidate cluster selection.

This design exploits the property that semantic similarity induces convergence in retrieval behavior. Given two query vectors v and v' with cosine similarity $d(v, v') \geq \delta$, their retrieved documents and probed clusters exhibit substantial overlap. We quantify this convergence using two metrics: top-k overlap $|R_k(v) \cap R_k(v')|$ and probe overlap $|C(v) \cap C(v')|$.

Empirical results (Figure 7) show that both metrics increase monotonically with inter-request similarity, indicating increasingly aligned retrieval behavior. As similarity increases, queries converge toward similar top-k results, while their ANN searches increasingly focus on shared clusters rather than exploring the full index. For example, on the NQ [35] dataset with $k = 3$ and $nprobe = 128$, the top-3 overlap approaches 2.37 when similarity exceeds 0.95, and similar queries share an average of 77.19 of the 128 probed clusters. Moreover, clusters containing top-k results consistently appear among the earliest probes. These results show that semantically similar requests can share retrieval computation, enabling similarity-guided reduction of search cost.

Leveraging this retrieval convergence, MNEMORAG realizes *similarity-aware retrieval* as described in Algorithm 1. A lightweight, capacity-bounded *Query Base* stores historical queries and their corresponding top-k results $R_k(v)$, capturing reusable retrieval signals without replicating the knowledge corpus. The *Query Base* employs LRU replacement, updating entry recency upon result reuse or similarity-guided retrieval. Queries served through direct reuse are not reinserted, while admitting a new query–result pair at capacity B evicts the least recently used entry. Given a new query v', the system identifies the most similar historical query v with the highest cosine similarity using efficient GPU-based matrix operations and obtains the corresponding similarity score s (Lines 3–4). Two similarity thresholds, τ_1 and τ_2, then determine the retrieval strategy according to s:

Top-k Result Reuse (Lines 5–6). If $s \geq \tau_1$, the high similarity between queries justifies directly reusing $R_k(v)$, thereby eliminating redundant probing. Based on empirical analysis, we set $\tau_1 = 0.95$ by default, reflecting that similarity scores above this threshold indicate sufficiently strong alignment between queries.

Adaptive Retrieval (Lines 7–13). If $\tau_2 \leq s < \tau_1$, retrieval is restricted to a similarity-adaptive probe space that balances efficiency and accuracy. We set $\tau_2 = 0.85$ by default to effectively leverage historical queries to guide the retrieval of new queries and maximize workload-level optimization. The probe budget is computed as:

$$real_nprobe(s) = \lceil n_{\min} + (n_{\max} - n_{\min})(1 - \tilde{s})^{\gamma} \rceil , \quad (1)$$

where

$$\tilde{s} = \frac{s - s_{\min}}{s_{\max} - s_{\min}} \quad (2)$$

is the normalized similarity based on the maximum and minimum similarity values maintained via the *Query Base*. Here, $n_{\min}$ and $n_{\max}$ denote the minimum and maximum bounds on the number of clusters to probe, and γ controls the decay rate of the function. The pruned cluster set is then defined as:

$$C_{\text{pruned}} = \{c_1, \ldots, c_{real_nprobe(s)}\}, \quad (3)$$

and augmented with high-quality clusters derived from the top-k results of the most similar query, denoted as $H(v)$:

$$P(v') = C_{\text{pruned}} \cup H(v), \quad (4)$$

ensuring retrieval is concentrated on semantically relevant regions while preserving coverage of high-quality candidate clusters. Typical parameter settings are $n_{\min} = 16$ and $n_{\max} = nprobe/2$, which empirically limit redundant cluster scans for highly similar queries while maintaining a sufficiently large probe budget to prevent accuracy degradation. The decay factor γ is set to 2, leveraging a convex reduction to accelerate

Algorithm 1 Similarity-Aware Retrieval

Variables: new query v'; *Query Base* storing historical queries and their top-k results; similarity thresholds τ_1, τ_2; probe budget bounds $n_{\min}, n_{\max}$; decay factor γ
Output: top-k results $R_k(v')$
$(v, R_k(v)) \leftarrow$ NEAREST_QUERY$(v', \text{Query Base})$
$s \leftarrow$ SIMILARITY(v, v')
if $s \geq \tau_1$ **then** /* Result reuse */
    $R_k(v') \leftarrow R_k(v)$
else if $s \geq \tau_2$ **then** /* Adaptive retrieval */
    $\tilde{s} \leftarrow$ NORMALIZE(s)
    $real_nprobe \leftarrow \lceil n_{\min} + (n_{\max} - n_{\min})(1 - \tilde{s})^{\gamma} \rceil$
    $C_{\text{pruned}} \leftarrow$ TOP_CLUSTERS$(v', real_nprobe)$
    $H(v) \leftarrow$ CLUSTERS_FROM_RESULTS$(R_k(v))$
    $P \leftarrow C_{\text{pruned}} \cup H(v)$
    $R_k(v') \leftarrow$ TOPK_SEARCH(v', P)
else /* Original retrieval */
    $R_k(v') \leftarrow$ TOPK_SEARCH$(v', \text{FULL_BUDGET})$
end if
UPDATE_QUERYBASE$(v', R_k(v'))$
return $R_k(v')$

the decrease of the probe budget for high-similarity queries, thus concentrating the search on more relevant clusters.

Original Retrieval (Lines 14–15). When the similarity score falls below the lower threshold τ_2, historical queries provide insufficient guidance for reliable search pruning, and the system defaults to full-budget probing to guarantee robust retrieval. In practice, as MNEMORAG operates over time, the *Query Base* accumulates historical queries, rendering this fallback increasingly infrequent.

C. Hotness-Driven Prefix Caching

MNEMORAG introduces *hotness-driven prefix caching*, a mechanism that exploits skewed document access patterns to reduce prefill overhead during generation. The key idea is to treat frequently retrieved documents as stable computational anchors across requests, and to organize prompt construction and KV cache management. To this end, MNEMORAG maintains a lightweight *Doc Base* that tracks a *hotness score* for accessed documents, using raw access frequency as a proxy for relative importance. This abstraction enables the system to prioritize hot documents in both prompt structure and KV cache retention, thereby aligning KV cache management with the workload distribution, while incurring negligible overhead.

To realize this design, MNEMORAG incorporates document hotness into prompt construction by querying the hotness scores of retrieved documents at an early stage of the generation process. As illustrated in Figure 8, based on their relative hotness, the system reorganizes these documents within the augmented prompt, placing frequently accessed ones earlier. Since prefix matching operates at the token level, such ordering increases the likelihood of cross-request prefix alignment, thereby improving KV cache reuse. However, this reordering may deviate from the original retrieval ranking

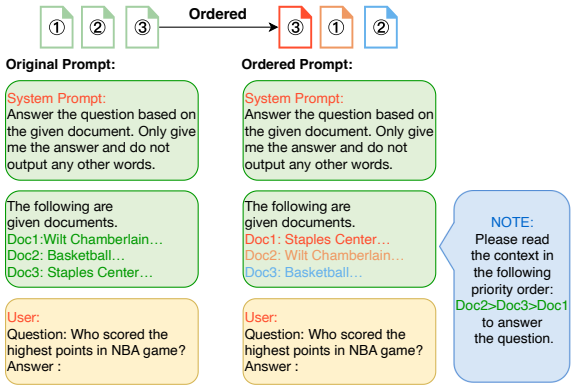


Fig. 8. Hotness-driven prompt construction.

and introduce the *lost-in-the-middle* effect [47], where content in the middle of the prompt receives disproportionately less attention. To address this, a lightweight *hint* encoding the original retrieval order is appended to the prompt [32], [65], [66]. This design preserves the retrieved context's semantic fidelity and interpretability while retaining the efficiency of hotness-driven prefix reordering.

While hotness-aware prompt construction maximizes cross-request prefix alignment, sustaining this benefit under limited GPU memory requires coordinated KV cache placement. To this end, MNEMORAG organizes prefix KV blocks in a hierarchical GPU–CPU cache. The GPU tier retains latency-critical prefixes, whereas the CPU tier serves as a higher-capacity backing store for reusable blocks displaced from GPU memory. This hierarchy extends prefix reuse beyond the capacity of the GPU cache while avoiding the prefill cost of recomputing previously materialized prefixes.

To govern GPU residency, MNEMORAG employs a *hotness-aware eviction policy* that accounts for both document hotness and recency. Prefix KV blocks that are frequently and recently reused are preferentially retained on the GPU, whereas stale or infrequently accessed blocks are selected for demotion. Formally, we operationalize this policy via a priority-based eviction scheme, where the priority of a KV cache block i is defined as:

$$\text{Priority}_i = \frac{H_i}{\log(1 + \Delta t_i)} \tag{5}$$

where H_i denotes the hotness score of the block's corresponding documents, reflecting its frequency of reuse, and $\Delta t_i > 0$ represents the elapsed time in seconds since its last access. The logarithmic scaling balances the influence of hotness and recency, gradually reducing the priority of stale blocks. To enforce this policy, MNEMORAG maintains priority-aware metadata for each KV cache block, enabling the selective eviction of low-priority blocks under GPU memory pressure.

Under GPU memory pressure, low-priority blocks are demoted to CPU memory rather than discarded. Upon a subsequent prefix match, these host-resident blocks are restored to the GPU before prefill, while recomputation is required only for blocks absent from both tiers. This placement pol-

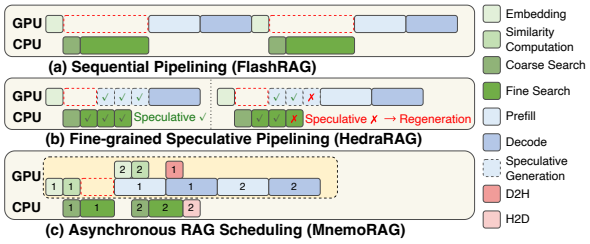


Fig. 9. Execution pipelines of conventional systems and MNEMORAG.

icy establishes a hot–warm–cold hierarchy: frequently reused blocks remain GPU-resident, less active but reusable blocks are retained in CPU memory, and stale blocks are removed when host capacity is exhausted. Although a CPU-cache hit incurs host-to-device transfer, it avoids redundant prefix computation and extends the effective KV cache capacity.

By combining KV cache management with hotness-aware prompt construction, MNEMORAG maximizes cross-request prefix reuse while adhering to GPU memory constraints. This design improves prefix KV cache hit rate, reduces redundant prefill computation, and preserves the efficiency and fidelity of retrieval-augmented generation.

D. Asynchronous RAG Scheduling

Sequential execution and speculative pipelining constitute two representative strategies for coordinating retrieval and generation. Figure 9(a) illustrates the stage-sequential execution of FlashRAG, whereas Figure 9(b) depicts the fine-grained speculative pipeline of HedraRAG [17], [34]. Figure 9(c) presents the asynchronous inter-request schedule of MNEMORAG. Rather than initiating generation before retrieval completes, MNEMORAG admits only retrieval-complete requests to generation while concurrently preparing subsequent requests, thereby achieving inter-request overlap without speculative outputs. Numerals 1 and 2 denote two consecutive requests, with the same scheduling discipline applied at microbatch granularity in the implementation.

MnemoRAG realizes this schedule through a serialized retrieval executor, a bounded generation queue, and a persistent vLLM `AsyncLLM` engine. Incoming requests are organized into microbatches, with a retrieval lock admitting one microbatch at a time to the shared Query Base and IVF index. After retrieval and prompt construction complete, the resulting requests are asynchronously submitted to the generation engine. Under a batch-level prefill-admission policy, retrieval for the next microbatch is released only after every request in the current microbatch has been admitted to prefill. This policy establishes a controlled one-microbatch look-ahead window, allowing subsequent retrieval to overlap with ongoing prefill and decoding without advancing arbitrarily ahead of generation. The bounded queue further limits outstanding generation microbatches and propagates backpressure to the retrieval executor, thereby constraining resource contention.

MnemoRAG also exploits intra-request parallelism along the preparation path. After GPU-based query embedding, historical-query similarity computation and IVF coarse search proceed concurrently on the GPU and CPU, respectively, as both consume the query embedding but are otherwise independent. Figure 9(c) depicts this overlap for each request. The similarity result determines the retrieval path and probing scope, whereas coarse search identifies the candidate clusters. Exact-reuse requests return cached neighbors directly, while guided- and full-search requests enter fine search after both branches complete. Consequently, the combined latency of similarity computation and coarse search is determined by the slower branch rather than their sum.

Cross-request overlap is activated once request 1 is admitted to generation. During its prefill and decoding stages, MNEMORAG prepares request 2: GPU-based embedding and similarity computation overlap with prefill, while CPU-based coarse and fine search extends across the prefill and decoding intervals. This schedule accounts for resource asymmetry. CPU retrieval can occupy the generation window, whereas the GPU retrieval operations are selectively admitted to limit contention with model inference. Prefix KV movement is incorporated into the pipeline. After prefill, eligible KV blocks from request 1 are offloaded through a dedicated D2H stream, while the reusable blocks required by request 2 are restored from pinned host memory through an independent H2D stream. As indicated by the transfer blocks labeled 1 and 2, both transfers proceed concurrently with the decoding of request 1.

CUDA events signal the completion of asynchronous KV transfers, and prefill for request 2 is released only after all required H2D operations finish. Generation remains ordered, while the retrieval and KV-restoration stages of request 2 proceed within the generation window of request 1. If these preparation stages complete before request 1 finishes decoding, request 2 enters prefill without an intervening pipeline stall. Combined with *similarity-aware retrieval*, which shortens the preparation path, this cross-request schedule allows retrieval and KV restoration to be more effectively absorbed within ongoing generation. The resulting event-driven execution preserves intra-request dependencies while coordinating three levels of concurrency: similarity computation with coarse search, subsequent retrieval with ongoing generation, and bidirectional KV transfer with decoding. Collectively, these mechanisms reduce exposed preparation latency and sustain more continuous GPU utilization.

V. EVALUATION

System. We evaluate MNEMORAG on a CPU–GPU hybrid server. Vector search runs on 64 physical cores of an AMD EPYC 7742 processor, with 512 GB of host memory for the vector index and offloaded KV caches. LLM generation is performed on eight NVIDIA A100 40 GB NVLink GPUs.

Models. Our primary model is Llama3-8B [67], served on a single GPU. To evaluate generality across model architectures and scales, we additionally include Qwen3-32B [68] and Llama3-70B [67], covering diverse LLM backbones.

Corpus and Index. We use Wikipedia [69] passages as the retrieval corpus (approximately 21M passages). Each passage is encoded into a 1024-dimensional dense vector using the

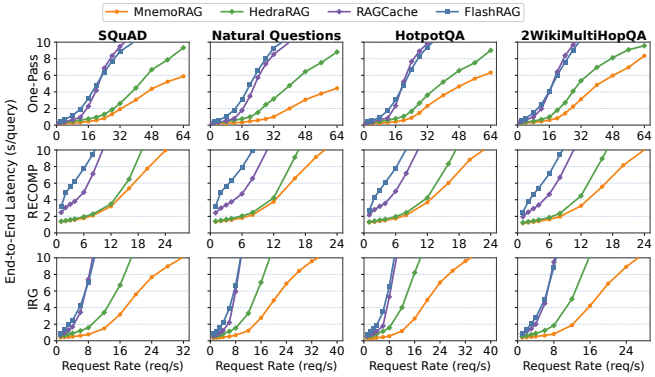


Fig. 10. End-to-end latency across datasets and RAG workflows.

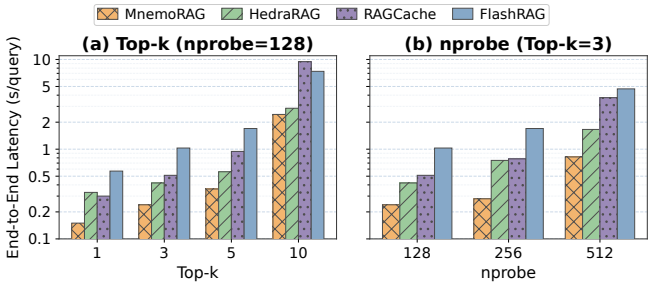


Fig. 11. Sensitivity of latency to retrieval depth and search breadth.

E5-large embedding model [42]. For efficient large-scale retrieval, we construct an IVF4096-Flat index, a widely adopted approximate nearest neighbor structure [55].

Datasets. We evaluate MNEMORAG on four widely used open-domain QA datasets: SQuAD [37], Natural Questions (NQ) [35], HotpotQA [36] and 2WikiMultiHopQA (2Wiki) [38]. For each dataset, we randomly sample 512 test queries for evaluation and use the same sample across all systems. These datasets cover passage comprehension, single-hop factoid QA and multi-hop reasoning workloads requiring bridging evidence, providing diverse evaluation scenarios.

RAG Workflows. We consider three representative RAG workflows: One-Pass [9], RECOMP [49], and iterative retrieval–generation (IRG) [53], corresponding to standard retrieval-then-generation, compression-augmented, and iterative retrieval–generation paradigms, respectively.

Baselines. For the end-to-end evaluation, we compare MNEMORAG with FlashRAG [17], RAGCache [22], and HedraRAG [34] across different RAG workflows. To isolate retrieval performance, we compare MNEMORAG with GPTCache [28], QVCache [29], HaS [26], and Proximity [27] in terms of retrieval latency, covering semantic result caching, query-aware vector caching, and optimized retrieval execution. We further evaluate generation-side cache reuse against RAGCache's PGDSF policy [22], ContextPilot [32], LRU [39], and FCFS [39], using the average and maximum prefix KV-cache hit rates as the primary metrics. For a fair comparison, all methods use identical configurations, including a maximum batch size of 64 and the same GPU memory allocation.

Metrics. We measure retrieval quality by Recall@k, the fraction of exact top-k neighbors recovered by ANN search, and generation quality by token-level F1 and answer accuracy (ACC), capturing answer overlap and correctness.

A. Overall Improvement

End-to-End Performance Evaluation. We first evaluate the end-to-end latency of MNEMORAG across four QA datasets and three representative RAG workflows under Poisson request arrivals at varying rates. As shown in Figure 10, MNEMORAG improves both latency efficiency and load scalability across datasets and workflows. Under One-Pass, it achieves average speedups of $1.54\times$–$2.22\times$ over HedraRAG, the strongest competing system, with the largest gain on NQ. This advantage carries over to multi-stage workflows, where the dataset-level average speedup reaches $1.26\times$ on RECOMP and $2.46\times$ on IRG. Beyond these average improvements, the pointwise speedup at matched request rates reaches $3.41\times$, $11.54\times$, and $11.73\times$ over HedraRAG, RAGCache, and FlashRAG, respectively. The latency reduction translates into greater serving capacity: under a 10 s mean-latency service-level objective (SLO) [34], MNEMORAG sustains $1.25\times$–$2.00\times$ the request rate of HedraRAG on RECOMP and IRG, and up to $4.00\times$ that of RAGCache and FlashRAG.

These efficiency gains entail only modest quality trade-offs. As shown in Table I, Recall@3 trails the strongest competitor by 1.67 percentage points on average, with gaps ranging from 1.1 to 2.5 points across settings. The reduction is most evident on SQuAD and negligible on 2Wiki. Nevertheless, answer quality remains stable: the average F1 and ACC gaps are only -0.07 and $+0.06$ percentage points, respectively, while MNEMORAG achieves the best or tied-best result in 14 of the 24 comparisons. These results indicate that approximate retrieval substantially reduces serving latency without materially compromising the evidence available for generation.

The consistency of these gains across workload content and execution structure demonstrates that the benefits of MNEMORAG are not confined to a specific dataset or pipeline configuration. Instead, they arise from eliminating redundant retrieval through cross-request semantic similarity, improving prefix reuse through hotness-aware KV-cache management, and overlapping retrieval with generation. By addressing complementary stages of RAG execution, these mechanisms jointly mitigate the principal sources of end-to-end latency.

Performance Across Retrieval Parameters. We further examine the sensitivity of end-to-end latency to the key retrieval parameters top-k and $nprobe$. As shown in Figure 11(a), increasing top-k raises latency across all systems because more retrieved documents enlarge the generation context and reduce opportunities for prefix KV reuse. Nevertheless, MNEMORAG achieves the lowest latency, increasing from 0.15 s at $k = 1$ to 2.44 s at $k = 10$, compared with 2.86 s, 9.45 s, and 7.37 s for HedraRAG, RAGCache, and FlashRAG at k=10. RAGCache's sharper rise is amplified as its larger knowledge-tree KV working set triggers victim-driven transfers beyond the overlap window, increasing queueing delay, whereas MNEMORAG limits this pressure through hotness-driven KV retention. Figure 11(b) shows that the advantage becomes more pronounced

TABLE I
RETRIEVAL AND GENERATION QUALITY ACROSS RAG SYSTEMS.

Dataset	Workflow	FlashRAG			RAGCache			HedraRAG			MnemoRAG		
		Recall@3	F1	ACC	Recall@3	F1	ACC	Recall@3	F1	ACC	Recall@3	F1	ACC
SQuAD	One-pass	0.976	0.469	0.469	**0.977**	0.471	**0.471**	0.942	0.442	0.441	0.952	**0.473**	**0.471**
	RECOMP	**0.977**	0.450	0.438	**0.977**	**0.453**	**0.443**	0.942	0.423	0.408	0.952	0.452	0.436
	IRG	**0.977**	**0.422**	0.439	**0.977**	**0.422**	0.439	0.942	0.392	0.412	0.956	0.420	**0.445**
NQ	One-pass	**0.988**	0.513	0.518	**0.988**	0.512	0.516	0.971	0.517	0.521	0.972	**0.518**	**0.523**
	RECOMP	**0.988**	0.516	0.510	**0.988**	0.509	0.504	0.971	0.510	0.506	0.973	**0.523**	**0.520**
	IRG	**0.987**	0.503	0.518	**0.987**	**0.504**	0.521	0.971	0.486	0.492	0.972	0.499	**0.523**
HotpotQA	One-pass	**0.968**	0.466	0.445	**0.968**	0.461	0.439	0.929	0.461	0.441	0.952	**0.468**	**0.449**
	RECOMP	**0.968**	0.418	0.398	**0.968**	**0.423**	**0.404**	0.929	0.416	0.396	0.953	**0.423**	0.402
	IRG	0.967	**0.457**	**0.430**	**0.969**	**0.457**	**0.430**	0.929	0.455	**0.430**	0.953	0.456	**0.430**
2Wiki	One-pass	**0.907**	**0.230**	**0.287**	**0.907**	0.228	0.285	0.856	0.223	0.281	0.894	0.228	0.281
	RECOMP	**0.907**	0.212	0.242	0.905	0.217	**0.250**	0.856	**0.221**	0.240	0.895	0.214	0.244
	IRG	**0.906**	0.202	0.232	**0.906**	0.196	0.225	0.856	0.189	0.213	0.895	**0.205**	**0.236**

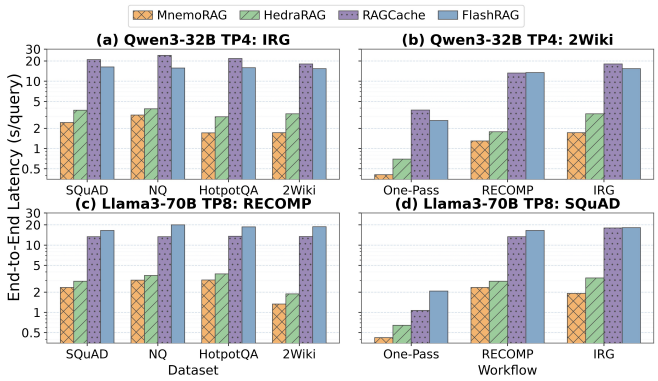


Fig. 12. End-to-end latency across larger LLM backbones.

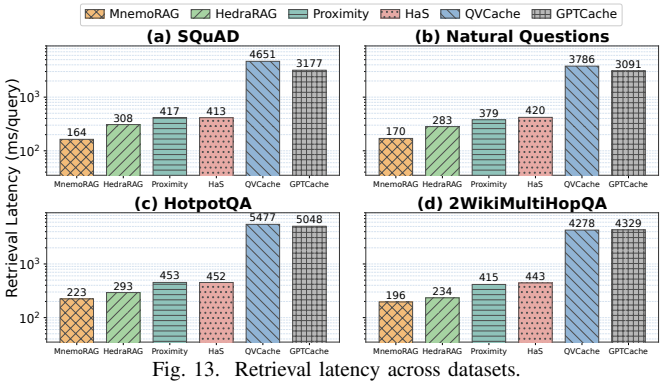


Fig. 13. Retrieval latency across datasets.

TABLE II
RETRIEVAL RECALL@3 ACROSS DATASETS.

System	Dataset			
	SQuAD	NQ	HotpotQA	2Wiki
GPTCache	0.221	0.273	0.774	0.266
QVCache	**0.976**	**0.988**	**0.969**	**0.906**
HaS	0.975	0.986	0.968	0.904
Proximity	0.975	0.987	**0.969**	0.905
HedraRAG	0.942	0.971	0.929	0.856
MnemoRAG	0.952	0.972	0.953	0.894

as $nprobe$ increases. At $nprobe = 512$, MNEMORAG requires only 0.82 s, whereas HedraRAG, RAGCache, and FlashRAG require 1.66 s, 3.75 s, and 4.70 s, respectively, corresponding to speedups of 2.02×–5.73×. Overall, these results demonstrate that MNEMORAG maintains a robust latency advantage under increasing retrieval complexity by amortizing redundant retrieval computation across semantically related requests and exploiting cross-request prefix KV reuse.

Performance Across LLM Backbones. We also evaluate the scalability of MNEMORAG using Qwen3-32B with TP4 and Llama3-70B with TP8. As shown in Figure 12, MNEMORAG delivers the lowest end-to-end latency across evaluated datasets and workflows, providing speedups of 1.24×–1.91× on Qwen3-32B and 1.17×–1.69× on Llama3-70B over HedraRAG. These results show that the latency advantage of MNEMORAG persists across model architectures and tensor-parallel configurations. As model scale increases, generation accounts for a larger fraction of the critical path because of greater computational demand, memory pressure, and inter-GPU synchronization overhead, thereby limiting the benefit attainable through retrieval optimization alone. Under this generation-dominated regime, performance increasingly depends on maximizing KV reuse and concealing request-preparation latency. MNEMORAG employs hotness-driven cache management to prioritize the retention of prefix KV states with high expected reuse utility, while asynchronous scheduling overlaps retrieval and KV restoration with generation. This combination reduces prefill overhead and prevents request preparation from extending the critical path, sustaining MNEMORAG's latency advantage as model scale increases.

B. Performance Analysis

Retrieval Efficiency. We evaluate retrieval efficiency across four datasets against five representative baselines with

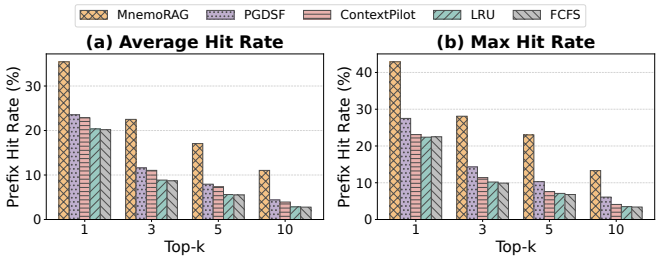


Fig. 14. Prefix KV cache reuse on NQ under varying top-k.

$nprobe = 128$ and QPS $= 8$. As shown in Figure 13, MNEMORAG consistently achieves the lowest latency, ranging from 164 to 223 ms/query. Relative to HedraRAG, the lowest-latency baseline, MNEMORAG provides a $1.19\times$–$1.88\times$ speedup, increasing to $2.03\times$–$2.54\times$ over Proximity and HaS. These gains incur only a limited reduction in retrieval quality: Table II shows that MNEMORAG achieves a strict Recall@3 of 0.894–0.972, remaining within 1.21–2.39 percentage points of the best result on each dataset. QVCache preserves high recall through region-specific hit validation, but cache misses still invoke the ANN search, resulting in $21.83\times$–$28.36\times$ the latency of MNEMORAG. GPTCache reuses results based on query-level semantic similarity, which does not guarantee exact top-3 neighbor consistency, resulting in low strict Recall@3. In contrast, MNEMORAG adapts the retrieval path and probing scope using inter-request similarity, achieving a more favorable balance between retrieval latency and quality.

Prefix KV Cache Hit Rate. We evaluate prefix KV cache reuse on NQ under varying top-k settings. Figure 14 shows that MNEMORAG consistently achieves the highest average and maximum hit rates across all top-k settings, reaching 35.4% and 42.9%, respectively, at top-$k = 1$. Compared with RAGCache's prefix-aware PGDSF policy, MNEMORAG achieves $1.51\times$–$2.51\times$ average hit rates and $1.56\times$–$2.23\times$ maximum hit rates, while consistently outperforming ContextPilot. As increasing k reduces cross-request prefix overlap, its average-hit-rate advantage over PGDSF grows from $1.51\times$ at top-$k = 1$ to $2.51\times$ at top-$k = 10$. Whereas PGDSF optimizes prefix-aware cache replacement and ContextPilot promotes reuse through context indexing, reordering, and deduplication, MNEMORAG integrates hotness-aware document ordering with two-tier KV-cache management to preserve reusable prefixes when prefix locality is limited.

Scheduling Efficiency. Figure 15 presents the batch-level latency breakdown under $nprobe = 128$ and top-$k = 1$. EB, SC, and CS denote embedding, similarity computation, and coarse search, while D2H and H2D denote KV Cache offloading from GPU to CPU and reloading from CPU to GPU, respectively. The baseline executes retrieval and generation sequentially, resulting in a 1,877.4 ms batch interval. In contrast, MNEMORAG overlaps the retrieval of batch $i+1$ with the generation of batch i. After EB, the 4.6 ms SC stage evaluates inter-request similarity to select the retrieval path concurrently with the 7.7 ms CS stage that probes the coarse index, allowing the SC latency to be fully hidden by CS. Guided by this similarity signal, *similarity-aware retrieval* reduces fine-search latency from 1,176.9 to 532.9 ms, while *hotness-driven prefix caching* reduces prefill latency from 370.9 to 304.1 ms through prefix KV reuse. Fine search and asynchronous D2H/H2D transfers are further overlapped with prefill and decoding, leaving the 615.2 ms generation stage as the steady-state pipeline bottleneck. Despite modest contention that increases EB latency from 16.4 to 19.6 ms and decode latency from 305.5 to 311.1 ms, reduced retrieval and prefill latency together with cross-batch pipelining lower batch interval from 1,877.4 to 615.2 ms, yielding a $3.05\times$ speedup.

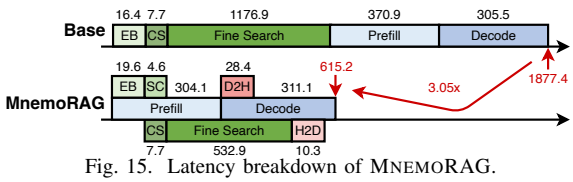


Fig. 15. Latency breakdown of MNEMORAG.

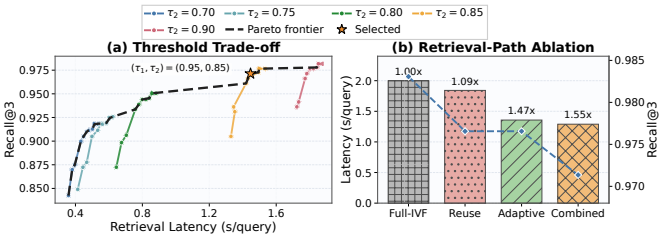


Fig. 16. Pareto similarity-threshold trade-off and retrieval-path ablation.

C. Ablation Studies

Similarity Thresholds and Path Ablation. We calibrate the two similarity thresholds on NQ: τ_1 governs top-k result reuse, while τ_2 activates adaptive retrieval. With $\tau_2 = 0.85$, raising τ_1 beyond 0.95 improves Recall@3 by 0.005 while increasing latency from 1.45 to 1.54 s; lower values sacrifice recall. With $\tau_1 = 0.95$, raising τ_2 from 0.85 to 0.90 similarly gains 0.005 Recall@3 but increases latency to 1.80 s. We therefore select the Pareto-efficient configuration $(\tau_1, \tau_2) = (0.95, 0.85)$, achieving 0.971 Recall@3 at 1.45 s. Path ablation confirms this trade-off: relative to Full-IVF at 0.983 Recall@3, Result Reuse and Adaptive Retrieval achieve $1.09\times$ and $1.47\times$ speedups at 0.977 Recall@3, while their combination reaches $1.55\times$ at 0.971 Recall@3. In practice, we calibrate (τ_1, τ_2) separately for each dataset using 256 requests, disjoint from the 512-request formal workload. A compact grid search selects latency–Recall@3 Pareto-optimal threshold pairs for formal evaluation. This yields $(0.94, 0.85)$ for SQuAD, $(0.93, 0.80)$ for HotpotQA, and $(0.96, 0.90)$ for 2Wiki.

Effect of Document Ordering. We evaluate three document-ordering strategies on 512 reordered requests per dataset from SQuAD and 2Wiki: original retrieval order, hotness-based reordering, and hotness-based reordering augmented with an ordering hint. As shown in Table III, Hotness+Hint improves both metrics over Hotness in every setting. On SQuAD, it achieves the highest ACC at all depths and the highest F1 at top-$k = 5$ and 10, with Original narrowly leading F1 at top-$k = 3$. For 2Wiki, its advantage is concentrated at top-$k = 3$ and 5; at top-$k = 10$, it leads only in ACC, while Original retains the highest F1. Overall, the ordering hint mitigates the

TABLE III
EFFECT OF DOCUMENT ORDERING STRATEGIES

Dataset	Top-k	Metric	Original	Hotness	Hotness+Hint
SQuAD	3	F1	**0.5279**	0.5246	0.5273
		ACC	0.5352	0.5332	**0.5410**
	5	F1	0.5044	0.5056	**0.5125**
		ACC	0.5488	0.5430	**0.5625**
	10	F1	0.4102	0.4153	**0.4172**
		ACC	0.5449	0.5645	**0.5664**
2Wiki	3	F1	0.1321	0.1309	**0.1463**
		ACC	0.1699	0.1641	**0.1719**
	5	F1	0.1039	0.0997	**0.1096**
		ACC	0.1641	0.1523	**0.1738**
	10	F1	**0.0921**	0.0704	0.0852
		ACC	0.1602	0.1523	**0.1621**

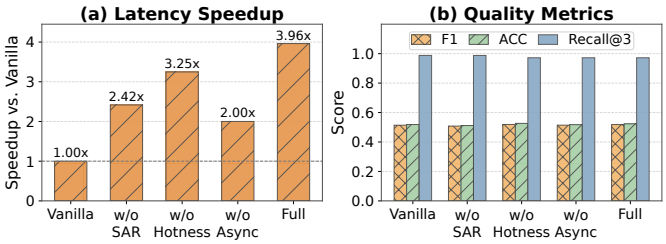


Fig. 17. Component ablation of MNEMORAG.

quality degradation associated with hotness-only reordering, whereas its advantage over the original retrieval order varies across datasets, retrieval depths, and evaluation metrics.

Component Ablation. We conduct a leave-one-out ablation on NQ at 8 QPS to quantify the contribution of each optimization in MNEMORAG. Vanilla RAG serves as the reference implementation, combining Faiss retrieval with vLLM generation without framework-level orchestration or MNEMORAG-specific optimizations. As shown in Figure 17, the Full configuration, which combines similarity-aware retrieval (SAR), hotness-driven prefix caching (Hotness), and asynchronous scheduling (Async), achieves a $3.96\times$ speedup over Vanilla. Removing Async reduces the speedup to $2.00\times$, while removing SAR and Hotness reduces it to $2.42\times$ and $3.25\times$, respectively, identifying Async as the largest contributor, followed by SAR and Hotness. Generation quality remains stable, with F1 ranging from 0.508 to 0.519 and ACC from 0.512 to 0.526. ANN Recall@3 decreases from approximately 0.988 for Vanilla and w/o SAR to 0.972 for the SAR-enabled configurations, reflecting the accuracy–efficiency trade-off of approximate retrieval without additional degradation from Hotness or Async. Collectively, SAR reduces retrieval computation, Hotness enhances prefix reuse, and Async shortens exposed preparation latency through cross-request stage overlap, allowing their combination to deliver the highest end-to-end speedup while maintaining comparable answer quality.

VI. RELATED WORK

Retrieval-Augmented Generation (RAG). RAG research spans algorithmic and system-level optimizations for efficiency and generation quality. METIS [70] adapts RAG configurations per query to balance latency and output fidelity. RAGCache [22] enables cross-query KV reuse through knowledge-aware caching but does not reorder retrieved documents to improve prefix reuse. TeleRAG [71] exploits intra-request similarity between original and refined queries to prefetch IVF clusters from CPU to GPU. PipeRAG [72] pipelines iterative retrieval and generation, while RAGO [33] optimizes RAG scheduling and resource management. HedraRAG [34] coordinates heterogeneous retrieval–generation workflows through graph-based sub-stage scheduling, whereas Omnia [73] speculatively overlaps coarse-to-fine reranking with progressive prefilling. In contrast, MNEMORAG jointly addresses retrieval redundancy and resource utilization across retrieval and generation to improve RAG serving efficiency.

Retrieval Optimization. Prior work reduces ANN search cost through query-adaptive search, result reuse, and tiered index placement. Query-adaptive methods tune search effort to query difficulty or target recall [74]–[76], while similarity-aware caches reuse results when similarity exceeds global or region-specific thresholds [26], [27], [29]. VectorLiteRAG [77] partitions IVF clusters across CPU and GPU tiers based on access skew and latency estimates. MNEMORAG instead uses inter-request similarity to guide *similarity-aware retrieval*, choosing between direct reuse and adaptive IVF search to reduce redundant computation while retaining relevant evidence.

Generation Optimization. KV caching is central to efficient LLM generation, with prior work following two directions. The first reduces the memory footprint through quantization [61], [63], [78]–[80], compression [81]–[85], and layer sharing [86]–[88], often at the cost of prediction accuracy. The second direction exploits *prefix KV cache reuse* [22], [39], [44]–[46], [61] to eliminate redundant computation across requests. However, exact-reuse methods remain constrained by prefix alignment [32], [39], [44], [46], [89], although cache-aware reordering extends reuse coverage [32], [89]. Approximate reuse relaxes this constraint, but incurs state-management and fidelity-control overheads [30], [45], [61], [90], [91]. MNEMORAG instead employs *hotness-driven prefix caching*, prioritizing frequently accessed documents to maximize cache utilization while preserving generation quality.

VII. CONCLUSION

We present MNEMORAG, a high-performance RAG system that achieves substantial efficiency gains by exploiting inter-request correlations. By integrating similarity-aware retrieval, hotness-driven prefix caching, and asynchronous RAG scheduling, MNEMORAG minimizes redundant vector computations, maximizes KV cache reuse, and alleviates cross-stage pipeline bottlenecks. Extensive evaluations across diverse RAG workflows and open datasets demonstrate that MNEMORAG consistently outperforms state-of-the-art systems, delivering significant speedups while maintaining retrieval recall and generation quality. These results underscore MNEMORAG's robustness, scalability, and practical effectiveness for high-performance RAG serving.

AI-GENERATED CONTENT ACKNOWLEDGMENT

ChatGPT (GPT-6) was used only for limited language refinement. It was not involved in the technical development or evaluation of this work. The authors take full responsibility for the manuscript.

REFERENCES

[1] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.

[4] S. Siriwardhana, R. Weerasekera, E. Wen, T. Kaluarachchi, R. Rana, and S. Nanayakkara, "Improving the domain adaptation of retrieval augmented generation (rag) models for open domain question answering," *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1–17, 2023.

[5] V. Magesh, F. Surani, M. Dahl, M. Suzgun, C. D. Manning, and D. E. Ho, "Hallucination-free? assessing the reliability of leading ai legal research tools," *Journal of empirical legal studies*, vol. 22, no. 2, pp. 216–242, 2025.

[6] A. Pandey, "Retrieval augmented fraud detection," in *Proceedings of the 5th ACM International Conference on AI in Finance*, 2024, pp. 328–335.

[7] C. Zelin, W. K. Chung, M. Jeanne, G. Zhang, and C. Weng, "Rare disease diagnosis using knowledge guided retrieval augmentation for chatgpt," *Journal of Biomedical Informatics*, vol. 157, p. 104702, 2024.

[8] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin *et al.*, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM transactions on information systems*, vol. 43, no. 2, pp. 1–55, 2025.

[9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in neural information processing systems*, vol. 33, pp. 9459–9474, 2020.

[10] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, and H. Hajishirzi, "When not to trust language models: Investigating effectiveness of parametric and non-parametric memories," in *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: Long papers)*, 2023, pp. 9802–9822.

[11] O. Ram, Y. Levine, I. Dalmedigos, D. Muhlgay, A. Shashua, K. Leyton-Brown, and Y. Shoham, "In-context retrieval-augmented language models," *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1316–1331, 2023.

[12] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. B. Van Den Driessche, J.-B. Lespiau, B. Damoc, A. Clark *et al.*, "Improving language models by retrieving from trillions of tokens," in *International conference on machine learning*. PMLR, 2022, pp. 2206–2240.

[13] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen *et al.*, "Siren's song in the ai ocean: A survey on hallucination in large language models," *Computational Linguistics*, vol. 51, no. 4, pp. 1373–1418, 2025.

[14] R. Shao, J. He, A. Asai, W. Shi, T. Dettmers, S. Min, L. Zettlemoyer, and P. W. Koh, "Scaling retrieval-based language models with a trillion-token datastore," *Advances in Neural Information Processing Systems*, vol. 37, pp. 91 260–91 299, 2024.

[15] B. Wang, W. Ping, L. McAfee, P. Xu, B. Li, M. Shoeybi, and B. Catanzaro, "Instructretro: Instruction tuning post retrieval-augmented pretraining," *arXiv preprint arXiv:2310.07713*, 2023.

[16] *LangChain*, https://github.com/langchain-ai/langchain, 2026.

[17] J. Jin, Y. Zhu, Z. Dou, G. Dong, X. Yang, C. Zhang, T. Zhao, Z. Yang, and J.-R. Wen, "Flashrag: A modular toolkit for efficient retrieval-augmented generation research," in *Companion Proceedings of the ACM on Web Conference 2025*, 2025, pp. 737–740.

[18] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The faiss library," *IEEE Transactions on Big Data*, 2025.

[19] J. Chen, H. Lin, X. Han, and L. Sun, "Benchmarking large language models in retrieval-augmented generation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, 2024, pp. 17 754–17 762.

[20] M. Shen, M. Umar, K. Maeng, G. E. Suh, and U. Gupta, "Towards understanding systems trade-offs in retrieval-augmented generation model inference," *arXiv preprint arXiv:2412.11854*, 2024.

[21] Y. Zhu, J.-C. Gu, C. Sikora, H. Ko, Y. Liu, C.-C. Lin, L. Shu, L. Luo, L. Meng, B. Liu *et al.*, "Accelerating inference of retrieval-augmented generation via sparse context selection," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 25 965–25 981.

[22] C. Jin, Z. Zhang, X. Jiang, F. Liu, S. Liu, X. Liu, and X. Jin, "Ragcache: Efficient knowledge caching for retrieval-augmented generation," *ACM Transactions on Computer Systems*, vol. 44, no. 1, pp. 1–27, 2025.

[23] B. Gao, Z. He, P. Sharma, Q. Kang, D. Jevdjic, J. Deng, X. Yang, Z. Yu, and P. Zuo, "{Cost-Efficient} large language model serving for multi-turn conversations with {CachedAttention}," in *2024 USENIX annual technical conference (USENIX ATC 24)*, 2024, pp. 111–126.

[24] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, "Retrieval augmented language model pre-training," in *International conference on machine learning*. PMLR, 2020, pp. 3929–3938.

[25] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, "From local to global: A graph rag approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[26] P. Peng, W. Lin, W. Wu, X. Wang, and Y. Liu, "Has: Accelerating rag through homology-aware speculative retrieval," *arXiv preprint arXiv:2604.20452*, 2026.

[27] S. Bergman, A.-M. Kermarrec, D. Petrescu, R. Pires, M. Randl, M. De Vos, and J. Zhang, "Leveraging approximate caching for faster retrieval-augmented generation," in *Proceedings of the 26th International Middleware Conference*, 2025, pp. 340–353.

[28] F. Bang, "Gptcache: An open-source semantic cache for llm applications enabling faster answers and cost savings," in *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*, 2023, pp. 212–218.

[29] A. E. Göçer, I. Tsakalidou, H. Nicholson, K. Kim, and A. Ailamaki, "Qvcache: A query-aware vector cache," *arXiv preprint arXiv:2602.02057*, 2026.

[30] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang, "Cacheblend: Fast large language model serving for rag with cached knowledge fusion," in *Proceedings of the twentieth European conference on computer systems*, 2025, pp. 94–109.

[31] S. Agarwal, S. Sundaresan, S. Mitra, D. Mahapatra, A. Gupta, R. Sharma, N. J. Kapu, T. Yu, and S. Saini, "Cache-craft: Managing chunk-caches for efficient retrieval-augmented generation," *Proceedings of the ACM on Management of Data*, vol. 3, no. 3, pp. 1–28, 2025.

[32] Y. Jiang, Y. Huang, L. Cheng, C. Deng, X. Sun, and L. Mai, "Contextpilot: Fast long-context inference via context reuse," *Proceedings of Machine Learning and Systems*, vol. 8, pp. 1639–1655, 2026.

[33] W. Jiang, S. Subramanian, C. Graves, G. Alonso, A. Yazdanbakhsh, and V. Dadu, "Rago: Systematic performance optimization for retrieval-augmented generation serving," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 974–989.

[34] Z. Hu, V. Murthy, Z. Pan, W. Li, X. Fang, Y. Ding, and Y. Wang, "Hedrarag: Co-optimizing generation and retrieval for heterogeneous rag workflows," in *Proceedings of the ACM SIGOPS 31st symposium on operating systems principles*, 2025, pp. 623–638.

[35] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee *et al.*, "Natural questions: a benchmark for question answering research," *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 453–466, 2019.

[36] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, "Hotpotqa: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of the 2018 conference on empirical methods in natural language processing*, 2018, pp. 2369–2380.

[37] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "Squad: 100,000+ questions for machine comprehension of text," in *Proceedings of the 2016 conference on empirical methods in natural language processing*, 2016, pp. 2383–2392.

[38] X. Ho, A.-K. D. Nguyen, S. Sugawara, and A. Aizawa, "Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps,"

in *Proceedings of the 28th International Conference on Computational Linguistics*, 2020, pp. 6609–6625.

[39] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th symposium on operating systems principles*, 2023, pp. 611–626.

[40] R. Dayarathne, U. Ranaweera, and U. Ganegoda, "Comparing the performance of llms in rag-based question-answering: A case study in computer science literature," in *International Conference on Artificial Intelligence in Education Technology*. Springer, 2025, pp. 387–403.

[41] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 2019, pp. 4171–4186.

[42] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei, "Text embeddings by weakly-supervised contrastive pre-training," *arXiv preprint arXiv:2212.03533*, 2022.

[43] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Multilingual e5 text embeddings: A technical report," *arXiv preprint arXiv:2402.05672*, 2024.

[44] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez *et al.*, "Sglang: Efficient execution of structured language model programs," *Advances in neural information processing systems*, vol. 37, pp. 62 557–62 583, 2024.

[45] I. Gim, G. Chen, S.-s. Lee, N. Sarda, A. Khandelwal, and L. Zhong, "Prompt cache: Modular attention reuse for low-latency inference," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 325–338, 2024.

[46] L. Ye, Z. Tao, Y. Huang, and Y. Li, "Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 11 608–11 620.

[47] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the association for computational linguistics*, vol. 12, pp. 157–173, 2024.

[48] G. Kim, S. Kim, B. Jeon, J. Park, and J. Kang, "Tree of clarifications: Answering ambiguous questions with retrieval-augmented large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 996–1009.

[49] F. Xu, W. Shi, and E. Choi, "Recomp: Improving retrieval-augmented lms with context compression and selective augmentation," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 43 478–43 502.

[50] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation," *arXiv preprint arXiv:2401.15884*, 2024.

[51] Y. Yu, W. Ping, Z. Liu, B. Wang, J. You, C. Zhang, M. Shoeybi, and B. Catanzaro, "Rankrag: Unifying context ranking with retrieval-augmented generation in llms," *Advances in neural information processing systems*, vol. 37, pp. 121 156–121 184, 2024.

[52] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, "Active retrieval augmented generation," in *Proceedings of the 2023 conference on empirical methods in natural language processing*, 2023, pp. 7969–7992.

[53] Z. Shao, Y. Gong, Y. Shen, M. Huang, N. Duan, and W. Chen, "Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 9248–9274.

[54] Z. Yue, H. Zhuang, A. Bai, K. Hui, R. Jagerman, H. Zeng, Z. Qin, D. Wang, X. Wang, and M. Bendersky, "Inference scaling for long-context retrieval augmented generation," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 72 914–72 938.

[55] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," *IEEE transactions on big data*, vol. 7, no. 3, pp. 535–547, 2019.

[56] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[57] A. Babenko and V. Lempitsky, "The inverted multi-index," *IEEE transactions on pattern analysis and machine intelligence*, vol. 37, no. 6, pp. 1247–1260, 2014.

[58] R. Yu, W. Huang, S. Bai, J. Zhou, and F. Wu, "Aquapipe: A quality-aware pipeline for knowledge retrieval and large language models," *Proceedings of the ACM on Management of Data*, vol. 3, no. 1, pp. 1–26, 2025.

[59] M. Ahmed, R. Seraj, and S. M. S. Islam, "The k-means algorithm: A comprehensive survey and performance evaluation," *Electronics*, vol. 9, no. 8, 2020.

[60] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "{DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 193–210.

[61] Y. Liu, H. Li, K. Du, J. Yao, Y. Cheng, Y. Huang, S. Lu, M. Maire, H. Hoffmann, A. Holtzman *et al.*, "Cachegen: Fast context loading for language model applications," *arXiv preprint arXiv:2310.07240*, 2023.

[62] Y. Li, R. Gu, C. Huan, Z. Wang, R. Yao, C. Tian, and G. Chen, "Hotprefix: Hotness-aware kv cache scheduling for efficient prefix sharing in llm inference systems," *Proceedings of the ACM on Management of Data*, vol. 3, no. 4, pp. 1–27, 2025.

[63] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "Flexgen: High-throughput generative inference of large language models with a single gpu," in *International Conference on Machine Learning*. PMLR, 2023, pp. 31 094–31 116.

[64] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for {Transformer-Based} generative models," in *16th USENIX symposium on operating systems design and implementation (OSDI 22)*, 2022, pp. 521–538.

[65] M. Zhang, Z. Meng, and N. Collier, "Can we instruct llms to compensate for position bias?" in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 12 545–12 556.

[66] F. Ye, S. Li, Y. Zhang, and L. Chen, "R2ag: Incorporating retrieval information into retrieval augmented generation," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 11 584–11 596.

[67] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[68] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[69] D. Chen, A. Fisch, J. Weston, and A. Bordes, "Reading wikipedia to answer open-domain questions," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2017, pp. 1870–1879.

[70] S. Ray, R. Pan, Z. Gu, K. Du, S. Feng, G. Ananthanarayanan, R. Netravali, and J. Jiang, "Metis: fast quality-aware rag systems with configuration adaptation," in *Proceedings of the ACM SIGOPS 31st symposium on operating systems principles*, 2025, pp. 606–622.

[71] C.-Y. Lin, K. Kamahori, Y. Liu, X. Shi, M. Kashyap, Y. Gu, R. Shao, Z. Ye, K. Zhu, R. Kadekodi *et al.*, "Telerag: Efficient retrieval-augmented generation inference with lookahead retrieval," *Proceedings of Machine Learning and Systems*, vol. 8, pp. 673–688, 2026.

[72] W. Jiang, S. Zhang, B. Han, J. Wang, B. Wang, and T. Kraska, "Piperag: Fast retrieval-augmented generation via algorithm-system co-design," *arXiv preprint arXiv:2403.05676*, 2024.

[73] R. Fu, S. Li, Y. Xu, T. Wu, Z. Ma, and J. Shi, "Omnia: Efficient rag serving through speculative scheduling," in *Proceedings of the 2026 The 35th International Symposium on High-Performance Parallel and Distributed Computing*, 2026, pp. 165–178.

[74] Z. Zhang, C. Jin, L. Tang, X. Liu, and X. Jin, "Fast, approximate vector queries on very large unstructured datasets," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 995–1011.

[75] S. Horchidan, F. Zeiher, H. Boström, and P. Carbone, "Conann: Conformal approximate nearest neighbor search," *Proceedings of the VLDB Endowment*, vol. 19, no. 1, pp. 29–42, 2025.

[76] M. Chatzakis, Y. Papakonstantinou, and T. Palpanas, "Darth: Declarative recall through early termination for approximate nearest neighbor search," *Proceedings of the ACM on Management of Data*, vol. 3, no. 4, pp. 1–26, 2025.

[77] J. Kim and D. Mahajan, "Vectorliterag: Latency-aware and fine-grained resource partitioning for efficient rag," in *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2026, pp. 1–15.

[78] H. Kang, Q. Zhang, S. Kundu, G. Jeong, Z. Liu, T. Krishna, and T. Zhao, "Gear: An efficient kv cache compression recipe for near-lossless generative inference of llm," *arXiv preprint arXiv:2403.05527*, 2024.

[79] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "Kivi: A tuning-free asymmetric 2bit quantization for kv cache," *arXiv preprint arXiv:2402.02750*, 2024.

[80] T. Zhang, J. Yi, Z. Xu, and A. Shrivastava, "Kv cache is 1 bit per channel: Efficient large language model inference with coupled quantization," *Advances in Neural Information Processing Systems*, vol. 37, pp. 3304–3331, 2024.

[81] M. Adnan, A. Arunkumar, G. Jain, P. J. Nair, I. Soloveychik, and P. Kamath, "Keyformer: Kv cache reduction through key tokens selection for efficient generative inference," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 114–127, 2024.

[82] Z. Liu, A. Desai, F. Liao, W. Wang, V. Xie, Z. Xu, A. Kyrillidis, and A. Shrivastava, "Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time," *Advances in Neural Information Processing Systems*, vol. 36, pp. 52 342–52 364, 2023.

[83] D. Yang, X. Han, Y. Gao, Y. Hu, S. Zhang, and H. Zhao, "Pyramidinfer: Pyramid kv cache compression for high-throughput llm inference," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 3258–3270.

[84] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett *et al.*, "H2o: Heavy-hitter oracle for efficient generative inference of large language models," *Advances in Neural Information Processing Systems*, vol. 36, pp. 34 661–34 710, 2023.

[85] Z. Li, E. Wallace, S. Shen, K. Lin, K. Keutzer, D. Klein, and J. Gonzalez, "Train big, then compress: Rethinking model size for efficient training and inference of transformers," in *International Conference on machine learning*. PMLR, 2020, pp. 5958–5968.

[86] Y. Xiong, H. Wu, C. Shao, Z. Wang, R. Zhang, Y. Guo, J. Zhao, K. Zhang, and Z. Pan, "Layerkv: Optimizing large language model serving with layer-wise kv cache management," *arXiv preprint arXiv:2410.00428*, 2024.

[87] J. Ainslie, J. Lee-Thorp, M. De Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "Gqa: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 4895–4901.

[88] N. Shazeer, "Fast transformer decoding: One write-head is all you need," *arXiv preprint arXiv:1911.02150*, 2019.

[89] K. Tan, R. Gu, and M. Li, "Cacheweaver: Cache-aware evidence ordering for efficient grounded rag inference," *arXiv preprint arXiv:2606.19667*, 2026.

[90] F. Fang, C.-H. Lo, Y. Liu, Y. Hua, and C. Qian, "Tessera: Demand-driven kv cache management for retrieval-augmented llm serving," *arXiv preprint arXiv:2609.32999*, 2026.

[91] X. Ma, D. Lee, A. Shah, and M. Jeon, "Cachesmith: Accurate rag serving under latency slos with adaptive kv reuse," *arXiv preprint arXiv:2610.18925*, 2026.